

Manuscript title: Genomic diversity reveals breeding pools in Brazilian tetraploid potato germplasm

Authors: Givanildo Rodrigues da Silva¹, Gustavo Nandi¹, Marcio F. R. Resende², Tiago de Souza Marçal³, Thiago Leandro Factor⁴, Fernando Angelo Piotto^{1*}

Affiliations:

¹ Genetics Department, Luiz de Queiroz College of Agriculture, University of São Paulo, Piracicaba, São Paulo, Brazil

² Horticultural Sciences Department, University of Florida, Gainesville, Florida, USA

³ Department of Biology, University of Lavras, Lavras, Minas Gerais, Brazil

⁴ Horticulture Center, Agronomic Institute (IAC), Campinas, São Paulo, Brazil

*Corresponding author: fpiotto@usp.br

A workflow to multilocus *FST* genome scan using dosage-derived allele counts in *poolfstat* package

Overview

This is proposed workflow and it was based on the *poolfstat* package (Gautier, 2026). Specific procedures and package algorithm specifications can be verified on its own manual (DOI: [10.32614/CRAN.package.poolfstat](https://doi.org/10.32614/CRAN.package.poolfstat); <https://CRAN.R-project.org/package=poolfstat>), which has also a vignette available at <https://cran.r-project.org/web/packages/poolfstat/vignettes/vignette.pdf>.

A differentiation-based genome scan was performed as a complementary approach for identifying putative genomic regions associated with divergent breeding histories or selection between Brazilian and foreign potato germplasm. The analysis used the R package *poolfstat* (Gautier et al., 2022), which implements *F*-statistic (called as *FST* or F_{ST}) estimators for both pooled-sequencing read counts and standard population allele-count data (Hivert et al., 2018; Gautier et al., 2022). In this study, individual tetraploid genotype dosages were aggregated into population-level allele counts and stored as a *poolfstat* *countdata* object. Therefore, the analysis was based on allele counts derived from individually genotyped clones.

All analyses described in the end of this file can be replicated and was optimized to run using the 'final_snp_qc_maf_missing_refalt.xlsx' and 'origin_genotypes.xlsx' provided in the github (https://github.com/giva-nildo/diversity_analyses_LMH).

Genotype data and population definition

Following quality control described on the manuscript, the working dataset contained 12,595 biallelic SNPs scored in 449 tetraploid potato clones. Genotypes were represented as alternative-allele dosages, with values of 0, 1, 2, 3 and 4 corresponding to the number of alternative allele copies carried by each clone. SNP metadata included chromosome, physical position, reference allele and alternative allele. Marker identifiers followed the CHROM_POS convention so that dosage rows could be aligned unambiguously with genomic coordinates and allele labels.

Clone origin was obtained from the sample metadata and used to define two groups, Brazilian and Foreign. The Foreign group included accessions introduced from several countries and the eight accessions for which a specific country of origin was unavailable. After removal of clone CBM16-16 because of excessive missing data, the final group sizes were 402 Brazilian and 47 Foreign clones.

Conversion of tetraploid dosages into population allele counts

For SNP l , clone j and population g , let $d(l, j, g)$ denote the called alternative-allele dosage, where $d(l, j, g)$ is an integer from 0 to 4. For each population and SNP, the number of called individuals, the alternative-allele count, the total number of sampled allele copies and the reference-allele count were calculated as follows:

$$ALT(l, g) = \sum_j d(l, j, g)$$

$$N(l, g) = 4 \times ncalled(l, g)$$

$$REF(l, g) = N(l, g) - ALT(l, g)$$

Thus, each non-missing tetraploid genotype contributed four allele copies. Missing genotype calls contributed neither allele counts nor denominator copies, allowing the total count $N(l, g)$ to vary among SNPs and populations. The two matrices supplied to *poolfstat* were *refallele.count* = REF and *total.count* = N. The corresponding alternative-allele frequency was $\frac{ALT(l, g)}{N(l, g)}$; exchanging the reference and alternative labels does not change *FST*.

Construction of *poolfstat* *countdata* object

Samples were matched between the dosage matrix and the origin metadata and reordered consistently. SNPs were then sorted by chromosome and physical position, which is required for a scan based on consecutive markers. Dosage values were checked to ensure that all non-missing observations were integers from 0 to 4. Reference and total allele-count matrices were generated with one row per SNP and one column per population. The SNP information table was aligned by marker identifier and contained the position, reference allele and alternative allele.

The allele counts and SNP metadata were combined in an S4 *countdata* object with two populations. The number of SNPs was verified against the row counts of the reference-count matrix, total-count matrix and SNP information table before *FST* estimation.

FST estimation

FST was estimated with *computeFST* using *method* = "Anova". This analysis-of-variance estimator partitions variation into among-population and within-population components while accommodating unequal allele sample sizes. In probability-of-identity notation, *FST* can be expressed as:

$$FST = \frac{Q_1 - Q_2}{1 - Q_2}$$

where Q_1 and Q_2 represent within-population and between-population probabilities of allelic identity, respectively.

For the non-hierarchical analysis implemented here, the SNP-specific analysis-of-variance estimator can be written as:

$$\widehat{FST}_l = \frac{MSP_l - MSG_l}{MSP_l + (nc_l - 1)MSG_l}$$

where MSP_l is the among-population mean-square component, MSG_l is the within-population gene mean-square component and nc_l is the effective sample-size coefficient at locus l .

Multilocus estimates were calculated as the ratio of summed numerator and denominator contributions across loci, rather than as the arithmetic means of SNP-specific *FST* values:

$$\widehat{FST}_w = \frac{\sum_{l \in W} (MSP_l - MSG_l)}{\sum_{l \in W} [MSP_l + (nc_l - 1)MSG_l]}$$

Genome-wide and SNP-specific values were first obtained without block-jackknife resampling (*nsnp.per.bjack.block* = 0). Consequently, the analysis did not generate block-jackknife standard errors or confidence intervals.

Sliding-window genome scan

A multilocus *FST* scan was performed with windows of 50 consecutive eligible SNPs. The *computeFST* implementation advances by half of the specified window size; therefore, consecutive windows were separated by 25 SNPs and overlapped by 50%. Windows were generated independently within each chromosome and never crossed chromosome boundaries. Chromosomes required more than 50 eligible SNPs to be scanned, and terminal markers that did not form a complete 50-SNP window were not included in an incomplete final window.

Because windows were defined by marker number rather than physical length, their spans in base pairs varied with SNP density. For each window, *poolfst* returned the chromosome, start position, end position, midpoint, cumulative plotting position and multilocus *FST* value. The physical boundaries corresponded to the positions of the first and fiftieth SNPs in the window.

Candidate-window definition and region merging

Candidate windows (*W*) were defined using the empirical upper tail of the genome-wide multilocus *FST* distribution. The 95th percentile was calculated across all non-missing window estimates, and windows with *FST* values greater than or equal to this threshold were retained:

$$T95 = \text{quantile}(\{FST_W\}, 0.95), \text{ candidate } W \text{ if } FST_W \geq T95$$

This procedure ranked windows with unusually high differentiation and was not interpreted as a formal significance test. Retained windows were ordered by chromosome and physical coordinates. Windows that overlapped or were contiguous on the same chromosome were merged into broader candidate regions. Each merged region was summarized by its chromosome, start and end coordinates, number of contributing windows, maximum *FST* and mean *FST*. SNPs were assigned to retained windows by chromosome-specific interval joins.

Computational implementation

The following reproducible workflow corresponds directly to the methodological stages described above.

This step loads the required packages, defines the expected input and output locations, records the tetraploid ploidy and fixes the window size at 50 SNPs.

```

library(poolfstat)
library(data.table)
library(readxl)

project_dir <- normalizePath(".", winslash = "/", mustWork = TRUE)
input_snp <- file.path(project_dir, "final_snp_qc_maf_missing_refalt.xlsx")
input_meta <- file.path(project_dir, "origin_genotypes.xlsx")
outdir <- file.path(project_dir, "selective_sweep_analyses",
                    "poolfstat", "poolfstat_out")
dir.create(outdir, recursive = TRUE, showWarnings = FALSE)

ploidy <- 4L
window_snps <- 50L
stopifnot(file.exists(input_snp), file.exists(input_meta))

```

The SNP table must contain marker_ID, CHROM, POS, REF and ALT, followed by one dosage column per clone. Values are converted to integers, and marker identifiers are required to be unique.

```

df_final <- as.data.table(read_excel(input_snp))

required_marker_cols <- c("marker_ID", "CHROM", "POS", "REF", "ALT")
stopifnot(all(required_marker_cols %in% names(df_final)))
stopifnot(!anyDuplicated(df_final$marker_ID))

metadata_cols <- intersect(names(df_final),
                           c("marker_ID", "CHROM", "POS", "ID", "REF", "ALT"))
sample_cols <- setdiff(names(df_final), metadata_cols)
stopifnot(length(sample_cols) > 0L)

dos <- copy(df_final[, c("marker_ID", sample_cols), with = FALSE])
for (sample_id in sample_cols) {
  set(dos, j = sample_id, value = as.integer(dos[[sample_id]]))
}

map <- unique(df_final[, .(
  marker_ID, CHROM = as.character(CHROM), POS = as.integer(POS),
  REF = as.character(REF), ALT = as.character(ALT)
)])

```

The metadata file is expected to contain Clone Name and Cluster by origin. The script renames these fields, retains genotyped samples with a population label, verifies the two expected groups and exports final sample counts.

```
origin <- as.data.table(read_excel(input_meta))
stopifnot(all(c("Clone Name", "Cluster by origin") %in% names(origin)))
setnames(origin, c("Clone Name", "Cluster by origin"), c("sample_ID", "pop"))

meta <- merge(
  data.table(sample_ID = sample_cols),
  unique(origin[, .(sample_ID, pop)]),
  by = "sample_ID", all.x = TRUE
)
meta <- meta[!is.na(pop)]
meta[, pop := as.character(pop)]

expected_pops <- c("Brazilian", "Foreign")
stopifnot(setequal(unique(meta$pop), expected_pops))
meta[, pop := factor(pop, levels = expected_pops)]
setorder(meta, pop, sample_ID)

population_summary <- meta[, .N, by = pop]
fwrite(population_summary, file.path(outdir, "population_summary.tsv"), sep = "\t")
print(population_summary)
```

Dosage columns are placed in exactly the same order as the metadata rows. The map is joined to the dosage table, SNPs are ordered within chromosomes, and the final matrix is checked for valid tetraploid dosages.

```
meta <- meta[sample_ID %in% sample_cols]
dos <- dos[, c("marker_ID", meta$sample_ID), with = FALSE]

dt <- map[dos, on = "marker_ID", nomatch = 0L]
stopifnot(nrow(dt) == nrow(dos))
setorder(dt, CHROM, POS)

X <- as.matrix(dt[, meta$sample_ID, with = FALSE])
storage.mode(X) <- "integer"

stopifnot(nrow(X) == nrow(dt), ncol(X) == nrow(meta))
stopifnot(all(X[!is.na(X)] >= 0L & X[!is.na(X)] <= ploidy))
stopifnot(!anyNA(dt$CHROM), !anyNA(dt$POS), !anyNA(dt$REF), !anyNA(dt$ALT))
```

For each group, the alternative count is the sum of dosages, the total count is four times the number of called clones and the reference count is their difference. The count matrices use one column per group.

```
pops <- expected_pops

make_group_counts <- function(group_name) {
  idx <- which(as.character(meta$pop) == group_name)
  stopifnot(length(idx) > 0L)
  called <- rowSums(!is.na(X[, idx, drop = FALSE]))
  alt <- rowSums(X[, idx, drop = FALSE], na.rm = TRUE)
  total <- ploidy * called
  ref <- total - alt
  list(called = called, alt = alt, total = total, ref = ref)
}

counts <- setNames(lapply(pops, make_group_counts), pops)
ref_mat <- do.call(cbind, lapply(counts, `[`, "ref"))
tot_mat <- do.call(cbind, lapply(counts, `[`, "total"))
colnames(ref_mat) <- colnames(tot_mat) <- pops
storage.mode(ref_mat) <- storage.mode(tot_mat) <- "integer"

stopifnot(all(ref_mat >= 0L), all(tot_mat >= 0L), all(ref_mat <= tot_mat))
```

This optional but recommended output makes the dosage-to-count transformation auditable and permits direct confirmation of group-specific denominators at every SNP.

```
count_export <- dt[, .(marker_ID, CHROM, POS, REF, ALT)]
for (g in pops) {
  count_export[, paste0(g, "_REF_count") := counts[[g]]$ref]
  count_export[, paste0(g, "_ALT_count") := counts[[g]]$alt]
  count_export[, paste0(g, "_total_count") := counts[[g]]$total]
  count_export[, paste0(g, "_called_clones") := counts[[g]]$called]
}
fwrite(count_export, file.path(outdir, "population_allele_counts.tsv"), sep = "\t")
```

The SNP information table is aligned directly from the ordered data object. The S4 *countdata* object contains the two count matrices, genomic coordinates and population names.

```
snp_info_pf <- data.frame(
  Contig = dt$CHROM,
  Position = as.integer(dt$POS),
  Ref = dt$REF,
  Alt = dt$ALT,
  stringsAsFactors = FALSE
)

cd <- new("countdata",
  npops = as.integer(length(pops)),
  nsnp = as.integer(nrow(ref_mat)),
  refallele.count = ref_mat,
  total.count = tot_mat,
  snp.info = snp_info_pf,
  popnames = pops
)

stopifnot(cd@nsnp == nrow(cd@refallele.count),
  cd@nsnp == nrow(cd@total.count),
  cd@nsnp == nrow(cd@snp.info))
validObject(cd)
saveRDS(cd, file.path(outdir, "poolfstat_countdata.rds"))
```

The analysis-of-variance estimator is used without block-jackknife resampling. The overall estimate, SNP-specific *FST* values and identity components are written separately.

```
fst_global <- computeFST(
  cd, method = "Anova", nsnp.per.bjack.block = 0, verbose = TRUE
)
saveRDS(fst_global, file.path(outdir, "fst_global.rds"))

fst_summary <- data.table(
  statistic = names(fst_global$Fst),
  value = as.numeric(fst_global$Fst)
)
fwrite(fst_summary, file.path(outdir, "fst_global.tsv"), sep = "\t")

snp_results <- cbind(
  dt[, .(marker_ID, CHROM, POS, REF, ALT)],
  as.data.table(fst_global$snp.Fstats),
  as.data.table(fst_global$snp.Q)
)
fwrite(snp_results, file.path(outdir, "fst_snp.tsv"), sep = "\t")
```

Supplying *sliding.window.size* = 50 makes *computeFST* use 50 consecutive eligible SNPs with its half-window step. The complete window table is retained before any threshold is applied.

```
fst_scan <- computeFST(
  cd, method = "Anova", sliding.window.size = window_snps,
  nsnp.per.bjack.block = 0, verbose = TRUE
)
saveRDS(fst_scan, file.path(outdir, "fst_scan_window50.rds"))

w <- as.data.table(fst_scan$sliding.windows.fvalues)
required_window_cols <- c(
  "Chr", "Start", "End", "MidPos", "CumMidPos", "MultiLocusFst"
)
stopifnot(nrow(w) > 0L, all(required_window_cols %in% names(w)))
fwrite(w, file.path(outdir, "fst_windows_nsnps_50.tsv"), sep = "\t")
```

The threshold and the number of retained windows are saved as metadata. The greater-than-or-equal comparison preserves any ties at the 95th percentile.

```
threshold_probability <- 0.95
thr <- as.numeric(quantile(
  w$MultiLocusFst, probs = threshold_probability, na.rm = TRUE
))

sig_win <- copy(w[!is.na(MultiLocusFst) & MultiLocusFst >= thr])
setorder(sig_win, Chr, Start, End)
sig_win[, window_id := .I]

threshold_metadata <- data.table(
  parameter = c("quantile", "threshold", "all_windows", "retained_windows"),
  value = c(threshold_probability, thr, nrow(w), nrow(sig_win))
)
fwrite(threshold_metadata,
  file.path(outdir, "fst_threshold_metadata.tsv"), sep = "\t")
fwrite(sig_win, file.path(outdir, "fst_candidate_windows.tsv"), sep = "\t")
```

The cumulative maximum end coordinate prevents nested or chained overlaps from being split incorrectly. Region identifiers are unique across chromosomes.

```
sig_merge <- copy(sig_win)
setorder(sig_merge, Chr, Start, End)

sig_merge[, previous_max_end := shift(cummax(End)), by = Chr]
sig_merge[, new_region := is.na(previous_max_end) |
              Start > previous_max_end + 1L]
sig_merge[, local_region := cumsum(new_region), by = Chr]
sig_merge[, merged_id := paste0(Chr, "_R", local_region)]

regions_merged <- sig_merge[, .(
  Start = min(Start), End = max(End), n_windows = .N,
  maxFst = max(MultiLocusFst, na.rm = TRUE),
  meanFst = mean(MultiLocusFst, na.rm = TRUE)
), by = .(merged_id, Chr)]
setorder(regions_merged, Chr, Start)
fwrite(regions_merged,
  file.path(outdir, "fst_candidate_regions_merged.tsv"), sep = "\t")
```

A chromosome-specific overlap join maps SNP coordinates to retained windows. The SNP-by-window table preserves provenance, whereas the unique table is used for counting and cross-method intersection.

```
snp_intervals <- dt[, .(
  marker_ID, Chr = CHROM, Pos = POS, Ref = REF, Alt = ALT,
  SNPStart = POS, SNPEnd = POS
)]
windows_for_join <- sig_win[, .(
  window_id, Chr, Start, End, MultiLocusFst
)]

setkeyv(windows_for_join, c("Chr", "Start", "End"))
hits <- foverlaps(
  snp_intervals, windows_for_join,
  by.x = c("Chr", "SNPStart", "SNPEnd"),
  by.y = c("Chr", "Start", "End"),
  type = "within", nomatch = 0L
)

candidate_snps_unique <- unique(hits[, .(marker_ID, Chr, Pos, Ref, Alt)])
fwrite(hits, file.path(outdir, "fst_candidate_snp_window_hits.tsv"), sep = "\t")
fwrite(candidate_snps_unique,
  file.path(outdir, "fst_candidate_snps_unique.tsv"), sep = "\t")
message("Unique poolfst candidate SNPs: ", nrow(candidate_snps_unique))
```

```
# =====
# 7) PLOT WINDOW SCAN
chr_col <- as.integer(factor(w$Chr))
png(file.path(outdir, paste0("fst_windows_nsnp_", window_snps, ".png")),
     width = 1100, height = 550, res = 150)
```

References

- Hivert, V., Leblois, R., Petit, E. J., Gautier, M. & Vitalis, R. Measuring genetic differentiation from Pool-seq data. *Genetics* 210, 315-330 (2018). <https://doi.org/10.1534/genetics.118.300900>
- Gautier, M., Vitalis, R., Flori, L. & Estoup, A. f-Statistics estimation and admixture graph construction with Pool-Seq or allele count data using the R package poolfstat. *Molecular Ecology Resources* 22, 1394-1416 (2022). <https://doi.org/10.1111/1755-0998.13557>
- poolfstat on CRAN. Official package page, reference manual and vignette. Current release observed during this audit: 3.1.0; this is not necessarily the version used for the manuscript analysis. <https://cran.r-project.org/web/packages/poolfstat/>
- Weir, B. S. & Cockerham, C. C. Estimating F-statistics for the analysis of population structure. *Evolution* 38, 1358-1370 (1984).

Limitations

The *FST* scan identifies genomic intervals with exceptional allele-frequency differentiation between the two defined origin groups. Such differentiation may arise from selection, but it can also reflect drift, relatedness, founder effects, admixture, breeding-program history, unequal group sizes or the broad composition of the Foreign category.

Aggregation of tetraploid dosages preserves population allele frequencies but removes individual-level genotype configurations and pedigree information. The estimator does not explicitly model polysomic inheritance, preferential pairing, double reduction or clone relatedness. In addition, the 50-SNP windows varied in physical length and the empirical 95th-percentile threshold did not provide a demographic or permutation-based null model. Candidate regions were therefore interpreted as putative and require validation using independent germplasm, phenotypic information or functional evidence.